

Software Design — Assignment 1

CRUD Operations — Technical Documentation

Svegler Alexa, Patcas Rares, Toader Sidonia

April 3, 2026

Contents

1	Introduction	3
2	Tech Stack	3
3	Application Architecture	3
3.1	N-Layer Architecture	3
3.2	Class Diagram — Entities	4
4	CRUD User Implementation	4
4.1	User Entity	4
4.2	UserRepository	5
4.3	UserService	5
4.4	UserController — REST Endpoints	6
5	CRUD Post Implementation	6
5.1	Post Entity	6
5.2	PostRepository	6
5.3	PostService	7
5.4	PostController — REST Endpoints	8
6	CRUD Comment Implementation	8
6.1	Comment Entity	8
6.2	CommentRepository	8
6.3	CommentService	9
6.4	CommentController — REST Endpoints	10
7	Testing	10
8	Conclusions	10

1 Introduction

This document details the design and implementation of the CRUD Operations (Create, Read, Update, Delete) for the core functionalities: User, Post, and Comment, within the Instagram clone project. Developed as part of the Software Design course, the application serves as a comprehensive social media ecosystem where users can share visual content, engage through comments, and interact via a voting system.

2 Tech Stack

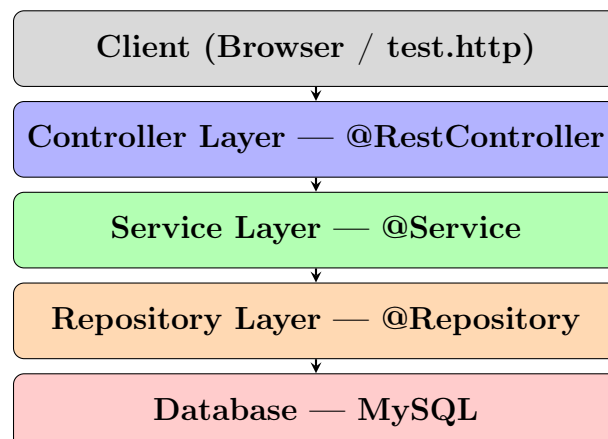
Technology	Version	Role
Java	17	Main programming language
Spring Boot	4.0.3	Main framework — auto configuration
Spring Web (MVC)	4.0.3	REST endpoint exposure
Spring Data JPA	4.0.3	Database access abstraction
Hibernate	7.2.4	JPA implementation — Java to SQL translation
Spring Security	4.0.3	Endpoint security
MySQL	8.0.45	Relational database
Lombok	1.18.30	Auto generation of getters/setters/constructors
Maven	3.x	Build tool and dependency management
Apache Tomcat	11.0.18	Embedded web server
JUnit 5	5.x	Testing framework

Table 1: Tech Stack used in the project

3 Application Architecture

3.1 N-Layer Architecture

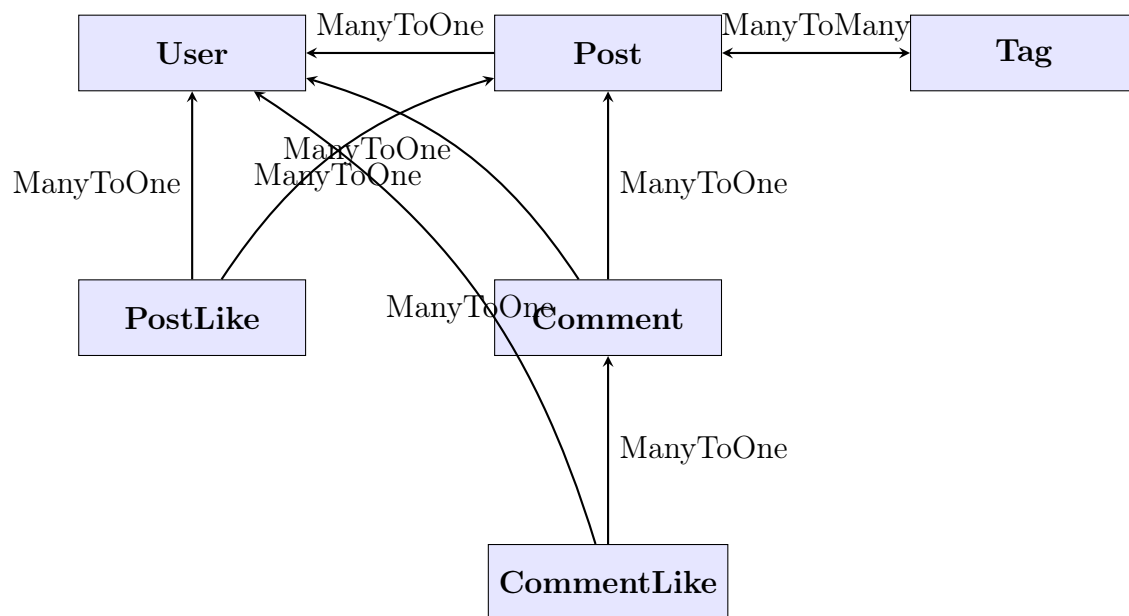
The application follows the **N-Layer (Layered Architecture)** pattern, with a clear separation between layers:



Layer	Annotation	Responsibility
Controller	@RestController	Receives HTTP requests, returns JSON responses
Service	@Service	Business logic, validations, rules
Repository	@Repository	Communication with the database
Entity	@Entity	Mapping Java classes to SQL tables

Table 2: Architecture layers

3.2 Class Diagram — Entities



4 CRUD User Implementation

4.1 User Entity

Listing 1: User.java

```

1 @Entity
2 @Table(name = "users")
3 @Data
4 @NoArgsConstructor
5 @AllArgsConstructor
6 public class User {
7
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long userId;
11
12    @Column(nullable = false)
13    private String username;
14
15    @Column(nullable = false)
16    private String password;
17

```

```
18     @Column(nullable = false)
19     private String email;
20
21     private String role; // "user" or "admin"
22     private Long score;
23     private boolean banned;
24 }
```

4.2 UserRepository

Listing 2: UserRepository.java

```
1 @Repository
2 public interface UserRepository extends JpaRepository<User, Long> { }
```

4.3 UserService

Listing 3: UserService.java

```
1 @Service
2 public class UserService {
3
4     @Autowired
5     private UserRepository userRepository;
6
7     public User createUser(User user) {
8         return userRepository.save(user);
9     }
10
11     public List<User> getAllUsers() {
12         return userRepository.findAll();
13     }
14
15     public Optional<User> getUserById(Long id) {
16         return userRepository.findById(id);
17     }
18
19     public User updateUser(Long id, User updatedUser) {
20         User existing = userRepository.findById(id)
21             .orElseThrow(() -> new RuntimeException("User not found"));
22         existing.setUsername(updatedUser.getUsername());
23         existing.setEmail(updatedUser.getEmail());
24         existing.setPassword(updatedUser.getPassword());
25         existing.setRole(updatedUser.getRole());
26         existing.setScore(updatedUser.getScore());
27         existing.setBanned(updatedUser.isBanned());
28         return userRepository.save(existing);
29     }
30
31     public void deleteUser(Long id) {
32         userRepository.deleteById(id);
33     }
34 }
```

4.4 UserController — REST Endpoints

HTTP	URL	Method	Description
POST	/users	createUser()	Creates a new user
GET	/users	getAllUsers()	Returns all users
GET	/users/{id}	getUserById()	Returns a user by ID
PUT	/users/{id}	updateUser()	Updates an existing user
DELETE	/users/{id}	deleteUser()	Deletes a user by ID

Table 3: REST Endpoints for User

5 CRUD Post Implementation

5.1 Post Entity

Listing 4: Post.java

```

1 @Entity
2 @Table(name = "posts")
3 @Data
4 @NoArgsConstructor
5 @AllArgsConstructor
6 public class Post {
7
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long postId;
11
12    @ManyToOne
13    @JoinColumn(name = "user_id")
14    private User author;
15
16    @Column(nullable = false)
17    private String title;
18    @Column(nullable = false)
19    private String text;
20    @Column(nullable = false)
21    private LocalDateTime date;
22    private String image;
23    private PostStatus status = PostStatus.JUST_POSTED;
24
25    @ManyToMany
26    @JoinTable(
27        name = "post_tags",
28        joinColumns = @JoinColumn(name = "post_id"),
29        inverseJoinColumns = @JoinColumn(name = "tag_id")
30    )
31    private List<Tag> tags;
32 }

```

5.2 PostRepository

Listing 5: PostRepository.java

```
1 @Repository
2 public interface PostRepository extends CrudRepository<Post, Long> {
3     List<Post> findAllByOrderByDateDesc();
4 }
```

5.3 PostService

Listing 6: PostService.java

```
1 @Service
2 public class PostService {
3
4     @Autowired
5     private PostRepository postRepository;
6
7     public Post addPost(Post post) {
8         return postRepository.save(post);
9     }
10
11     public List<Post> getAllPosts() {
12         return postRepository.findAllByOrderByDateDesc();
13     }
14
15     public Post getPostById(Long id) {
16         return postRepository.findById(id)
17             .orElseThrow(() -> new RuntimeException("Post not found"));
18     }
19
20     public Post updatePost(Post post, Long userId) {
21         Post existingPost = postRepository.findById(post.getId())
22             .orElseThrow(() -> new RuntimeException("Post not found"));
23         if (!existingPost.getAuthor().getUserId().equals(userId)) {
24             throw new RuntimeException("You are not allowed to update this post");
25         }
26         existingPost.setTitle(post.getTitle());
27         existingPost.setText(post.getText());
28         existingPost.setDate(post.getDate());
29         existingPost.setImage(post.getImage());
30         return postRepository.save(existingPost);
31     }
32
33     public void deletePost(Long id, Long userId) {
34         Post existingPost = postRepository.findById(id)
35             .orElseThrow(() -> new RuntimeException("Post not found"));
36         if (!existingPost.getAuthor().getUserId().equals(userId)) {
37             throw new RuntimeException("You are not allowed to delete this post");
38         }
39         postRepository.delete(existingPost);
40     }
41 }
```

5.4 PostController — REST Endpoints

HTTP	URL	Method	Description
POST	/posts	addPost()	Creates a new post
GET	/posts	getAllPosts()	Returns all posts
GET	/posts/{id}	getPostById()	Returns a post by ID
PUT	/posts/{id}	updatePost()	Updates a post
DELETE	/posts/{id}	deletePost()	Deletes a post

Table 4: REST Endpoints for Post

6 CRUD Comment Implementation

6.1 Comment Entity

Listing 7: Comment.java

```

1 @Entity
2 @Table(name = "comments")
3 @Data
4 @NoArgsConstructor
5 @AllArgsConstructor
6 public class Comment {
7
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long commId;
11
12    @ManyToOne
13    @JoinColumn(name="user_id")
14    private User author;
15
16    @ManyToOne
17    @JoinColumn(name="post_id")
18    private Post post;
19
20    @Column(nullable = false)
21    private String text;
22
23    private String image;
24
25    @Column(nullable = false)
26    private LocalDateTime date = LocalDateTime.now();
27 }

```

6.2 CommentRepository

Listing 8: CommentRepository.java

```

1 @Repository
2 public interface CommentRepository extends CrudRepository<Comment, Long>
3 {
4     List<Comment> findByPostPostId(Long postId);
5 }

```

6.3 CommentService

Listing 9: CommentService.java

```

1 @Service
2 public class CommentService {
3
4     @Autowired
5     private CommentRepository commentRepository;
6     @Autowired
7     private PostRepository postRepository;
8     @Autowired
9     private UserRepository userRepository;
10
11     public Comment createComment(Comment comment, Long postId, Long
        userId) {
12         User author = userRepository.findById(userId)
13             .orElseThrow(() -> new RuntimeException("User not found"
14             ));
15         Post post = postRepository.findById(postId)
16             .orElseThrow(() -> new RuntimeException("Post not found"
17             ));
18         if ((comment.getText() == null || comment.getText().trim().
19             isEmpty()) &&
20             (comment.getImage() == null || comment.getImage().isEmpty())
21             ) {
22             throw new RuntimeException("Comment must have text or image"
23             );
24         }
25         comment.setAuthor(author);
26         comment.setPost(post);
27         return commentRepository.save(comment);
28     }
29
30     public List<Comment> retrieveComments(Long postId) {
31         return commentRepository.findByPostId(postId);
32     }
33
34     public Comment updateComment(Comment updatedComment, Long userId) {
35         Comment existing = commentRepository.findById(updatedComment.
36             getCommId())
37             .orElseThrow(() -> new RuntimeException("Comment not
38             found"));
39         if (!existing.getAuthor().getUserId().equals(userId)) {
40             throw new RuntimeException("You are not allowed to edit this
41             comment");
42         }
43         existing.setText(updatedComment.getText());
44         existing.setImage(updatedComment.getImage());
45         return commentRepository.save(existing);
46     }
47
48     public void deleteComment(Long commentId, Long userId) {
49         Comment comment = commentRepository.findById(commentId)
50             .orElseThrow(() -> new RuntimeException("Comment not
51             found"));
52         if (!comment.getAuthor().getUserId().equals(userId) &&
53             !comment.getPost().getAuthor().getUserId().equals(userId)) {

```

```

45         throw new RuntimeException("You are not allowed to delete_
           this comment");
46     }
47     commentRepository.deleteById(commentId);
48 }
49
50 public Comment getCommentById(Long commentId) {
51     return commentRepository.findById(commentId)
52         .orElseThrow(() -> new RuntimeException("Comment not_
           found"));
53 }
54 }

```

6.4 CommentController — REST Endpoints

HTTP	URL	Method	Description
POST	/comments	createComment()	Creates a new comment
GET	/comments/post/{postId}	getPostComments()	Returns all comments for a specific post
GET	/comments/{commentId}	getComment()	Returns a comment by ID
PUT	/comments/{commentId}	editComment()	Updates a comment
DELETE	/comments/{commentId}	deleteComment()	Deletes a comment

Table 5: REST Endpoints for Comment

7 Testing

Integration tests were implemented using **JUnit 5** and **Spring Boot Test**.

Test	Operation	Result
createUser_ shouldSaveUserInDB	CREATE	Passed
getAllUsers_ shouldReturnList	READ ALL	Passed
getUserById_ shouldReturnUser	READ ONE	Passed
updateUser_ shouldModifyUser	UPDATE	Passed
deleteUser_ shouldRemoveUser	DELETE	Passed
contextLoads	Context	Passed

Table 6: Test results — 6/6 passed

All **6 tests** passed successfully in **823ms**.

8 Conclusions

- **Entity Mapping:** 6 entities correctly mapped with JPA relations.
- **CRUD User:** all 5 operations implemented and functional.
- **CRUD Post:** all operations implemented with ownership verification.

- **CRUD Comment:** all operations implemented with text/image validation.
- **Testing:** 6 integration tests, all passed successfully.
- **Security:** Spring Security configuration in place.
- **Clean Code:** layered architecture, naming conventions respected.